

Build a Sentiment Analyser in Python

A beginner's guide to Natural Language Processing (NLP) and rule-based sentiment detection.

1. Spark & Hook: Can Computers Read Feelings?

Imagine reading reviews on a food delivery app like **SnackDart** before ordering your lunch. When you read a review, your brain instantly determines whether the customer was happy or upset:

- "Samosa was tasty!" → **POSITIVE**
- "Dosa was cold." → **NEGATIVE**
- "Idli was okay." → **NEUTRAL**

How can we teach a computer—which only understands numbers—to evaluate text and understand human feelings like joy, disappointment, or neutrality?

2. Key Concept: Sentiment Analysis & Polarity Scoring

Sentiment Analysis is a field of Artificial Intelligence and Natural Language Processing (NLP) that trains computers to identify and extract opinions from text.

How Polarity Scoring Works

Computers assign a numerical score (known as **polarity score**) to specific emotion-driven words:

Sentiment Category	Example Words / Phrases	Score Assigned
Positive	"delicious", "tasty", "superb", "masto", "amazing"	+1
Neutral	"arrived on time", "okay", "food"	0
Negative	"cold", "soggy", "burnt", "bakwas", "boring"	-1

The total sentiment score of a full review is calculated by summing up the scores of all individual words contained in the sentence.

3. Text Normalization: Preparing Data for Processing

Computers strictly differentiate between uppercase and lowercase letters (e.g., "Tasty" is not equal to "tasty"). To ensure accurate word matching against our sentiment dictionaries, we perform **Text Normalization** using two essential Python string methods:

1. `.lower()`: Converts all characters in the string into lowercase letters.
2. `.split()`: Breaks the full sentence into individual words based on spaces, returning a list of words.

Transformation Example:

```
# Raw user input with mixed case
user_input = "Paneer Butter Masala"

# Step 1: Lowercase conversion
user_input = user_input.lower() # Result: "paneer butter masala"

# Step 2: Splitting into a list of words
words = user_input.split()      # Result: ["paneer", "butter", "masala"]
```

4. Sentiment Analyser Program Flowchart

Below is the step-by-step logic our Python program executes from start to finish:

Step 1: Input Review — Accept user text review.

Step 2: Lowercase Text — Convert text using `.lower()`.

Step 3: Split into Words — Tokenize into a word list using `.split()`.

Step 4: Loop through Words — Iterate through each word in the list using a for loop.

Step 5: Check Word Polarity — Check if the word exists in `positive_words` or `negative_words`.

Step 6: Calculate Score — Add +1 for positive words and subtract 1 for negative words.

Step 7: Display Result — Output whether overall sentiment is POSITIVE, NEGATIVE, or NEUTRAL.

5. Step-by-Step Code Implementation

Step 1: Define Word Dictionaries

Create lists containing positive and negative words, including local slang (e.g., "masto", "bakwas"):

```
positive_words = ["superb", "masto", "tasty", "amazing", "blockbuster", "great"]
negative_words = ["bakwas", "boring", "bad", "cold", "burnt", "flop"]
```

Step 2: Process Input & Calculate Score

Capture input, normalize it, and iterate over every word to compute the total sentiment score:

```
review = input("Enter your review: ")
review = review.lower()
words = review.split()

score = 0
for word in words:
    if word in positive_words:
        score += 1
    elif word in negative_words:
        score -= 1
```

Step 3: Classify & Output Results

Evaluate the total score using conditional statements:

```
if score > 0:
    print("Your review is POSITIVE 😊")
elif score < 0:
    print("Your review is NEGATIVE 😞")
else:
    print("Your review is NEUTRAL 😐")
```

Practice Questions

Test your understanding of sentiment analysis concepts and Python code implementation.

Question 1: Text Normalization

MULTIPLE CHOICE

What will be the output of the following Python code snippet?

```
text = "Samosa WAS Tasty"  
words = text.lower().split()  
print(words)
```

1. ["Samosa", "WAS", "Tasty"]
2. ["samosa", "was", "tasty"]
3. "samosa was tasty"
4. ["samosa", "was", "Tasty"]

Question 2: Manual Scoring Calculation

SHORT ANSWER

Given the word lists from the lesson:

- positive_words = ["superb", "masto", "tasty", "amazing", "blockbuster", "great"]
- negative_words = ["bakwas", "boring", "bad", "cold", "burnt", "flop"]

Calculate the numerical score and determine the final sentiment result for the following review:

"The food was masto but the dosa was cold and bad"

Question 3: Enhancing Dictionary Lists

CODE COMPLETION

Write the Python code to add two new positive words ("delicious" and "mindblowing") and two new negative words ("yuck" and "slow") to the existing lists using Python's list `.append()` or `.extend()` methods.

Question 4: Handling Negation (Advanced Challenge)

LOGIC & REFLECTION

Consider the review: *"The food was not tasty."*

1. What total score will our simple sentiment analyser calculate for this review? Why?
2. Why is this score incorrect, and how might negation words like "not " confuse a simple keyword-matching sentiment analyser?

Question 5: Movie Review Analyser

CODING CHALLENGE

Write a complete Python program that asks the user to input a movie review, converts the text to lowercase, splits it into words, and checks whether the review is positive, negative, or neutral using a custom word list tailored for movie reviews (e.g., *"hit"*, *"masterpiece"* vs *"flop"*, *"waste"*).